CovertScope Full Test & Improvement Report

Generated 2026-10-30 | Dashboard, API, database persistence, Docker runtime, dependency health, and browser E2E

OVERALL: PASS WITH FOLLOW-UP

The core reported behavior and reliability issues are fixed, and the complete frontend, backend, and browser test tiers pass. Browser testing found a real PostgreSQL type mismatch that prevented simulator flow persistence; the INET mapping and serialization were corrected and covered by a PostgreSQL regression test. Live interface capture remains a deployment/lab validation item.

Validation results

Area	Result	Evidence / limitation
Frontend unit tests	PASS	Vitest: 6 files, 24 tests passed; exit code 0.
Production build	PASS	Vite build succeeded; no oversized-chunk warning.
Frontend dependency audit	PASS	npm audit reported 0 vulnerabilities.
Backend full suite	PASS	18 tests passed plus 4 PostgreSQL subtests; exit code 0.
PostgreSQL persistence	PASS	INET flow IP ingest/read-back regression test passed; blocklist persistence verified.
Browser E2E	PASS	Playwright: 4 tests passed serially (auth, alert acknowledgement, blocklist, capture/dashboard).
Runtime/API	PASS	Docker Compose DB/backend healthy; browser tests exercised dashboard and authenticated API.
Native detector	PASS / LIMITED	Prior offline smoke classified a generated packet. No real network capture was run.

What is working well

- Demo traffic is paced predictably (browser demo around 3 seconds per flow; backend simulator waits 2-4 seconds), with synthetic alerts on a steady cadence instead of random bursts.

- Demo history starts small and retains an intentional 300-flow cap; throughput is calculated from elapsed flow time, and acknowledged high/critical alerts leave the active count.

- Protected API data requires authentication. The browser suite verifies invalid-key rejection and a successful valid-key connection.

- Dashboard lazy loading reduces the main JavaScript chunk. PostgreSQL migrations preserve existing data, and the Alembic logging configuration preserves application loggers.

Fixes completed in this validation

- Modeled PostgreSQL INET flow address columns correctly, bound parsed addresses as IP address objects, and serialized them back to API strings. This resolves simulator persistence failures and is regression-tested.

- Added Playwright browser coverage for authentication, alert acknowledgement, blocklist persistence, and capture/dashboard behavior. The suite runs with one worker to avoid shared-backend interference.

- Updated Playwright to use its managed Chromium browser and documented the E2E setup in README.

- Preserved application loggers when Alembic configures logging; the PostgreSQL-enabled backend suite passes after the fix.

Remaining follow-up and future development

- Validate all four detector signatures through live capture on an authorized, isolated Linux lab VM. No real live-interface capture was performed for this report.

- Configure matching COVERTSCOPE_INGEST_SECRET values before connecting the native engine to a deployed backend.

- Alert acknowledgement and flow/stat dashboard rendering use deterministic browser fixtures; they verify the browser journey but do not validate persistence for those API resources. The blocklist flow and capture start/stop use the real backend API.

- Migrate Pydantic class-based Config to ConfigDict before upgrading to Pydantic 3; three existing deprecation warnings remain.

- Run the PostgreSQL integration and Playwright suites in CI so database and browser regressions remain covered.

Security and test-data handling

No API keys, passwords, or other credentials are included. Earlier simulator test attempts created 69 synthetic flow rows and 2 associated alerts; after confirming the original database flow count was zero, those test rows and the reserved-IP browser fixture were removed. Final API read-back confirmed 0 flows, 0 alerts, and capture stopped. Browser E2E now uses fixtures for dashboard flow rendering and does not create synthetic flow rows.

Verdict: frontend tests/build, dependency scan, PostgreSQL-enabled backend suite, and browser E2E all pass. Remaining work is limited to authorized live-capture validation, secret configuration for deployment, and Pydantic deprecation cleanup.